COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	9/30/2026	**DATE SUBMITTED:**	9/30/2026
NAME:	Demain, Jaime Luis Esguerra, Paula Esteta, Jesse Matthew Estorco, Clint Hoven Fernandez, Cedric	**INSTRUCTOR:**	Engr. Jimlord Quejado

SEATWORK 10.1 Graph

1. PROCEDURE OUTPUT

1. Define the following: (10)

A. Graph

- A mathematical structure consisting of a set of objects (vertices or nodes) where some pairs of objects are connected by links (edges).

B. Vertex

- A fundamental unit or point that forms a graph, representing an individual entity or data point.

C. Nodes

- An alternative term for vertices, commonly used in computer science and network contexts to signify a point of connection.

D. Weight

- A numerical value assigned to an edge in a graph, often representing cost, distance, capacity, or resistance between two connected vertices.

E. Path

- A sequence of edges that connects a sequence of distinct vertices, allowing traversal from a starting vertex to a destination vertex.

F. Directed graph

- A graph where the edges have a specific orientation or direction, meaning they point

from one vertex to another (often represented by arrows).

G. Connected graph

- A graph in which there is a path between every pair of vertices, meaning no vertex or group of vertices is completely isolated.

H. Directed cyclic

- A directed graph that contains at least one cycle—a path that starts and ends at the same vertex by following the directed edges.

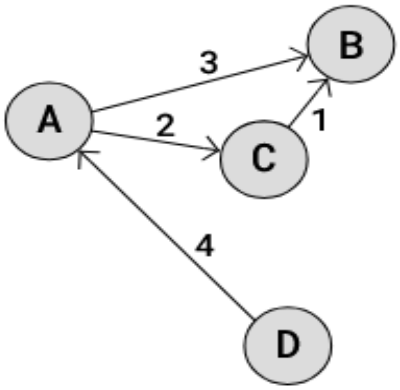
I. Adjacency matrix

- A two-dimensional square matrix used to represent a graph, where the rows and columns represent vertices, and the cell values indicate whether an edge exists between them (and its weight, if applicable).

J. Adjacency List

- A collection of unordered lists used to represent a graph, where each vertex has a dedicated list that stores all the other vertices it is directly connected to.

2. Identify the parts of the following graph. (10)



- Nodes: {A, B, C, D}

Edges	Weight
A to B	3
A to C	2
C to B	1
D to A	4

3. Plot the adjacency matrix of the graph above. (10 pts)

	A	B	C	D
A	0	3	2	0
B	0	0	0	0
C	0	1	0	0
D	4	0	0	0

4. Create a program in C++ program to create a graph that looks like in the above figure.

Code:

```
#include <iostream>
#include <vector>
#include <iomanip>

using namespace std;

int main() {
   // Number of nodes (A=0, B=1, C=2, D=3)
   const int NUM_NODES = 4;
   char nodeLabels[] = {'A', 'B', 'C', 'D'};

   // Adjacency matrix based on item 3
   // Rows/Columns: A, B, C, D
   int adjMatrix[NUM_NODES][NUM_NODES] = {
      {0, 3, 2, 0}, // Edges from A
      {0, 0, 0, 0}, // Edges from B
      {0, 1, 0, 0}, // Edges from C
      {4, 0, 0, 0}  // Edges from D
   };

   // Print Nodes
   cout << "Nodes: {A, B, C, D}\n\n";

   // Print Directed Edges and Weights (from item 2)
   cout << "Edges and Weights:\n";
   cout << "-------------------\n";
   for (int i = 0; i < NUM_NODES; i++) {
      for (int j = 0; j < NUM_NODES; j++) {
         if (adjMatrix[i][j] != 0) {
```

```
            cout << nodeLabels[i] << " to " << nodeLabels[j]
                 << " -> Weight: " << adjMatrix[i][j] << "\n";
        }
    }
}

cout << "\n";

// Print Adjacency Matrix (from item 3)
cout << "Adjacency Matrix:\n";
cout << "    A  B  C  D\n";
cout << "  -------------\n";
for (int i = 0; i < NUM_NODES; i++) {
    cout << nodeLabels[i] << " |";
    for (int j = 0; j < NUM_NODES; j++) {
        cout << setw(3) << adjMatrix[i][j];
    }
    cout << "\n";
}

return 0;
}
```

5. Differentiate a BFS vs DFS types of search.

Breadth-First Search (BFS)

- **Traversal Strategy:** Explores the graph level by level, visiting all immediate neighbor nodes before moving deeper into the graph.
- **Data Structure:** Uses a **Queue** (FIFO — First-In, First-Out).
- **Shortest Path:** Guarantees finding the shortest path in an unweighted graph.
- **Memory Usage:** Typically higher memory overhead ($O(V)$) because it must store all nodes at the current depth level in memory.
- **Primary Applications:** Finding the shortest path, peer-to-peer networks, and web crawlers.

Depth-First Search (DFS)

- **Traversal Strategy:** Explores as far as possible along each branch or path before backtracking to explore alternative paths.
- **Data Structure:** Uses a **Stack** (LIFO — Last-In, First-Out) or function call recursion.
- **Shortest Path:** Does not guarantee finding the shortest path.
- **Memory Usage:** Generally lower memory overhead ($O(H)$, where H is the maximum graph depth) since it only stores nodes along the current search path.

- **Primary Applications:** Cycle detection in graphs, topological sorting, and solving mazes/puzzles.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.